4주차 과제: 오늘의 할 일 - 코드 상세 해설

작성한 핵심 파일 5개(types.ts, TodoItem.tsx, TodoItem.module.css, App.tsx, App.module.css) 통합 해설서

1. 전체 파일 구조 및 역할 요약

- src/types.ts : 여러 파일에서 함께 쓰는 Todo 인터페이스와 Filter 리터럴 유니언 타입을 정의하고 export 합니다.
- src/components/TodoItem.tsx : 할 일 '하나'를 그리는 자식 컴포넌트입니다. 부모로부터 todo 데이터와 이벤트 핸들러(onToggle, onEdit, onDelete)를 전달받아 렌더링합니다.
- src/components/TodoItem.module.css : 개별 할 일 항목의 레이아웃과 완료 상태(.done)일 때의 취소선·흐린 글자 스타일을 정의합니다.
- src/App.tsx : 할 일 목록(todos)과 필터(filter) 상태를 관리하고, 추가·완료·수정·삭제·정렬 핸들러(handle~) 및 useEffect 브라우저 탭 제목 동기화를 수행합니다.
- src/App.module.css : 전체 페이지 레이아웃, 요약 바, 선택된 필터 탭(.activeTab), 빈 목록 안내, 하단 지우기 버튼 스타일을 정의합니다.

2. src/types.ts — 공통 타입 분리

```
export interface Todo {
  id: number;
  text: string;
  done: boolean;
}

export type Filter = "all" | "active" | "done";
```

- 별도 파일 분리 및 import type : App.tsx와 TodoItem.tsx에서 공통으로 참조하므로 별도 .ts 파일에 정의했습니다. 사용하는 곳에서는 import type으로 불러와 타입 전용 임포트 규칙을 지켰습니다.
- Todo 인터페이스 : id(숫자), text(문자열), done(불리언) 속성을 가지며, 자식 컴포넌트의 Props 타입(TodoItemProps)과 혼동되지 않게 이름을 구분했습니다.
- Filter 리터럴 유니언 타입 : 'all' | 'active' | 'done' 세 가지 문자열 리터럴만 허용하여 오타나 잘못된 필터 값을 컴파일 단계에서 방지합니다.

3. src/components/TodoItem.tsx & TodoItem.module.css

```
interface TodoItemProps {
  todo: Todo;
  onToggle: (id: number) => void;
  onEdit: (todo: Todo) => void;
  onDelete: (id: number) => void;
}

export default function TodoItem({ todo, onToggle, onEdit, onDelete }: TodoItemProps) {
  return (
    <li className={`${styles.todo} ${todo.done ? styles.done : ""}`}>
      <button type="button" onClick={() => onToggle(todo.id)}>
        {todo.done ? "*" : "o"}
      </button>
      <span className={styles.text}>{todo.text}</span>
      <button type="button" onClick={() => onEdit(todo)}>수정</button>
      <button type="button" onClick={() => onDelete(todo.id)}>삭제</button>
    </li>
  );
}
```

- Props 인터페이스 및 네이밍 : 자식 컴포넌트 props는 interface(TodoItemProps)로 정의하고, 부모의 상태 변경 함수(setTodos) 대신 로직을 담은 함수를 onToggle, onEdit, onDelete 이름으로 받습니다.
- 조건부 클래스 적용 : 백틱() 템플릿 리터럴과 삼항 연산자로 todo.done이 true일 때 styles.done 클래스를 추가하여 취소선(line-through)과 흐린 색상을 적용합니다.
- 완료 표시 : 완료된 할 일은 버튼 안에 '*'가, 미완료 할 일은 '○'가 표시되며 클릭 시 onToggle(todo.id)를 호출합니다.

4. src/App.tsx — 상태, 파생 값, useEffect (상반부)

```
const [todos, setTodos] = useState<Todo[]>([]);
const [filter, setFilter] = useState<Filter>("all");

const doneCount = todos.filter((todo) => todo.done).length;
const activeCount = todos.length - doneCount;
const isAllDone = todos.length > 0 && doneCount === todos.length;
const filteredTodos = todos.filter((todo) => {
  if (filter === "active") return !todo.done;
  if (filter === "done") return todo.done;
  return true;
});

useEffect(() => {
  document.title = activeCount > 0 ? `할 일 ${activeCount}개 남음` : "오늘의 할 일";
}, [activeCount]);
```

• 제네릭 타입 명시 : 빈 배열 []만으로는 요소 타입을 알 수 없으므로 useState<Todo[]>([])로, 필터 상태는 유니언 타입을 유지하도록 useState<Filter>('all')로 제네릭을 명시했습니다.

• 렌더링 중 파생 값 계산 : 완료 개수(doneCount), 남은 개수(activeCount), 모두 완료 여부(isAllDone), 현재 탭에 보여줄 목록(filteredTodos)은 별도 useState로 만들지 않고 렌더링 중에 계산하여 상태 불일치를 원천 차단합니다.

• useEffect 브라우저 탭 제목 변경 : 남은 할 일 개수(activeCount)가 1개 이상이면 '할 일 N개 남음', 0개이면 '오늘의 할 일'로 document.title을 변경하며, 의존성 배열에는 필요한 값인 [activeCount]만 넣었습니다.

5. src/App.tsx — 이벤트 핸들러와 불변성 (중반부)

```
const handleAdd = () => {
  const input = window.prompt("새로운 할 일을 입력하세요");
  if (input === null) return;
  const trimmed = input.trim();
  if (!trimmed) return;
  setTodos((prev) => [...prev, { id: Date.now(), text: trimmed, done: false }]);
  setFilter("all");
};

const handleToggle = (id: number) => {
  setTodos((prev) => prev.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
};

const handleEdit = (targetTodo: Todo) => {
  const input = window.prompt("수정할 내용을 입력하세요", targetTodo.text);
  if (input === null || !input.trim()) return;
  const trimmed = input.trim();
  setTodos((prev) => prev.map((t) => (t.id === targetTodo.id ? { ...t, text: trimmed } : t)));
};

const handleDelete = (id: number) => setTodos((prev) => prev.filter((t) => t.id !== id));
const handleSort = () => setTodos((prev) => [...prev].sort((a, b) => a.text.localeCompare(b.text, "ko")));
const handleClearDone = () => setTodos((prev) => prev.filter((t) => !t.done));
```

• window.prompt() 입력 검증 : 취소(null)를 누르거나 공백만 입력(!input.trim())한 경우 아무 동작도 하지 않습니다. 수정 시에는 두 번째 인자로 기존 내용(targetTodo.text)을 미리 채워줍니다.

• 불변성 유지 및 콜백 업데이트 : 원본 배열에 push, splice, sort를 직접 호출하지 않고, 항상 setTodos((prev) => ...) 콜백 형태와 [...prev], map, filter, [...prev].sort()를 사용하여 새로운 배열을 반환합니다.

6. src/App.tsx & App.module.css — 조건부 렌더링 및 스타일

• if / else if 빈 목록 안내 변수 : 할 일이 아예 없을 때(2줄 안내), '진행 중' 탭이 비었을 때, '완료' 탭이 비었을 때의 문구를 if / else if 문으로 emptyMessage 변수에 담아 사용합니다.

• 삼항 연산자 & AND(&&) 연산자 : 빈 목록 안내 ↔ 할 일 목록(<ul>) 전환은 삼항 연산자로 처리하고, 축하 문구(isAllDone && ...)와 완료 지우기 버튼(doneCount > 0 && ...)은 불리언 조건식과 && 연산자를 사용해 화면에 숫자 0이 출력되지 않게 했습니다.

• 선택된 필터 탭 스타일(App.module.css) : className={${styles.tab} }로 활성 탭에만 흰색 배경과 그림자를 부여합니다.